A

Report on

"SMS Spam Detection Using NLP"

---Name of Students---

Arnav Kalge (Roll No :-1)

Rohan Saswade (Roll No :-47)

Avishkar Panmand (Roll No :-24)

(SDG Goal No.) Under the

Guidance of

krushna jagtap

Assistant Professor, AI & DS Department

[TY B.Tech. Artificial Intelligence & Data Science]

AY (2026-2027)

At

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

AISSMS IOIT Pune

Affiliated to

Abstract:-

The SMS Spam Detection system is a Python-based mini-project that uses Natural Language Processing (NLP) and machine learning to classify a text message as spam or not spam (ham). The main objective is to automatically filter unwanted, promotional and fraudulent SMS messages. The system is trained on the SMS Spam Collection dataset from the UCI Machine Learning Repository, which contains 5,572 labelled messages. Text is converted into numerical features using TF-IDF (Term Frequency – Inverse Document Frequency) with English stop-word removal, and a Multinomial Naive Bayes classifier is trained on these features.

On a held-out test set of 1,115 messages the model achieved an accuracy of 97.04%, with a precision of 1.00 for spam (no legitimate message was wrongly marked as spam). The program also accepts a screenshot of a message: Tesseract OCR extracts the text from the image, the user confirms it, and it is then classified. Results are shown with accuracy, a classification report and a confusion matrix. The system is simple and fast, but it depends on an old, English-only dataset, and its spam recall (78%) shows that some spam messages are missed. It can be extended with newer datasets, multilingual support, and stronger models.

Index :-

Sr.no	Content	Page No
1.	Aim	1
2.	Objectives	1
3.	Theory	1-2
4.	Literature review	2-3
5.	Python code	3-5
6.	Output	6-7
7.	Conclusion	7
8.	References	7

1. AIM

To develop a Python-based SMS Spam Detection system using Natural Language Processing (NLP) that classifies a text message, typed or extracted from a screenshot, as Spam or Not Spam.

2. OBJECTIVES

- To understand the use of NLP and TF-IDF for text classification.
- To train a Multinomial Naive Bayes classifier on the SMS Spam Collection dataset.
- To evaluate the model using accuracy, precision, recall and a confusion matrix.
- To read messages from screenshots using Tesseract OCR and warn the user about spam.

3. THEORY

3.1 Introduction

Natural Language Processing (NLP) enables computers to process and understand human language. Spam detection is a binary text-classification problem: every SMS is labelled ham (not spam, 0) or spam (1). The project uses Python, pandas, scikit-learn (TF-IDF, Naive Bayes, metrics), Tesseract OCR (pytesseract), Pillow and tkinter (file picker).

3.2 Dataset

The UCI SMS Spam Collection has 5,572 messages: 4,825 ham (86.6%) and 747 spam (13.4%). It is split 80% / 20% into 4,457 training and 1,115 testing messages. The split is stratified (random_state = 42) so both sets keep the same ham-to-spam ratio.

3.3 Working Principle

The system follows these steps:

- Load the dataset and convert labels to numbers (ham = 0, spam = 1).
- Split the data into training and testing sets.
- Convert text to lowercase, remove English stop words and build TF-IDF vectors. The vectorizer is fitted on training data only, to avoid data leakage.
- Train the Multinomial Naive Bayes model and evaluate it on the test set.
- For a new message (typed, or read from a screenshot by OCR), apply the same vectorizer and predict Spam or Not Spam.

3.4 TF-IDF

TF-IDF gives a high weight to words that are frequent in one message but rare across all messages (for example “free”, “win”, “claim”).

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

3.5 Multinomial Naive Bayes

Multinomial Naive Bayes applies Bayes' theorem, assuming words are independent given the class. The message is assigned to the class with the higher probability. It is fast and works well on short text.

$$P(class \mid message) \propto P(class) \times P(w1 \mid class) \times P(w2 \mid class) \times \ldots \times P(wn \mid class)$$

3.6 Metrics

Accuracy = correct predictions / all predictions. Precision = real spam among messages predicted spam. Recall = spam messages caught out of all real spam. F1-score = harmonic mean of precision and recall.

4. LITERATURE REVIEW

Author(s) and Year	Research / Contribution	Relevance to the Project
Salton and Buckley (1988)	Studied term-weighting for text retrieval, the basis of TF-IDF.	Feature weighting used here.
McCallum and Nigam (1998)	Compared Naive Bayes event models for text classification.	Supports Multinomial Naive Bayes for text.
Almeida et al. (2011)	Built the SMS Spam Collection and compared spam filters on it.	Source of our dataset.
Pedregosa et al. (2011)	Introduced the scikit-learn library.	Vectorizer, model and metrics.
Smith (2007)	Described the Tesseract OCR engine.	Reads text from screenshots.

The literature shows that spam can be filtered with statistical and deep-learning methods. TF-IDF with Naive Bayes was chosen because it is simple, fast, explainable and accurate on SMS data.

5. PYTHON CODE

Program: SMS Spam Detection Using NLP

```
import pandas as pd
import pytesseract
from PIL import Image
from tkinter import Tk, filedialog
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. Load dataset
try:
    data = pd.read_csv("SMSSpamCollection", sep="\t", header=None,
                       names=["label", "message"], encoding="utf-8")
except FileNotFoundError:
    print("ERROR: Dataset file 'SMSSpamCollection' not found.")
    raise SystemExit(1)

print("Total messages:", len(data))

# 2. Prepare data
data["label"] = data["label"].map({"ham": 0, "spam": 1})
X_train, X_test, y_train, y_test = train_test_split(
    data["message"], data["label"], test_size=0.2,
    random_state=42, stratify=data["label"])

print("Training messages:", len(X_train))
print("Testing messages:", len(X_test))

# 3. TF-IDF vectorization
vectorizer = TfidfVectorizer(lowercase=True, stop_words="english")
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

# 4. Train model
model = MultinomialNB()
model.fit(X_train_tfidf, y_train)

# 5. Model performance
predictions = model.predict(X_test_tfidf)
print(f"Accuracy: {accuracy_score(y_test, predictions) * 100:.2f}%")
print(classification_report(y_test, predictions,
      target_names=["Not Spam", "Spam"], zero_division=0))
print(confusion_matrix(y_test, predictions))

# 6. Prediction function
def check_message(message):
    if not message.strip():
        print("No readable text was provided.")
        return
    prediction = model.predict(vectorizer.transform([message]))[0]
    if prediction == 1:
        print("Result: SPAM MESSAGE")
        print("Warning: This message may be unwanted.")
    else:
        print("Result: NOT SPAM MESSAGE")
        print("The model classified this as legitimate.")

# 7. Main menu
```

```
while True:
    print("\n1. Check a message\n2. Quit\n3. Upload a screenshot")
    choice = input("Enter your choice (1, 2, or 3): ").strip()
    if choice == "1":
        check_message(input("Enter your message: ").strip())
    elif choice == "2":
        print("Exiting program. Goodbye!")
        break
    elif choice == "3":
        root = Tk()
        root.withdraw()
        root.attributes("-topmost", True)
        image_path = filedialog.askopenfilename(
            title="Select a screenshot",
            filetypes=[("Image files", "*.png *.jpg *.jpeg *.webp"),
                       ("All files", "*.*")])
        root.destroy()
        if not image_path:
            print("No screenshot selected. Returning to menu.")
            continue
        try:
            with Image.open(image_path) as image:
                message = pytesseract.image_to_string(image).strip()  # OCR
            if not message:
                print("No readable text found in the screenshot.")
                continue
            print("Extracted text:\n" + message)
            confirm = input("Classify this text? (y/n): ").strip().lower()
            if confirm == "y":
                check_message(message)
            else:
                print("Classification cancelled.")
        except Exception as error:
            print("Could not process screenshot:", error)
    else:
        print("Invalid choice. Please enter 1, 2, or 3.")
```

6. OUTPUT

Sample Output Format

```
Total messages: 5572
Training messages: 4457
Testing messages: 1115
Accuracy: 97.04%

              precision    recall  f1-score   support

   Not Spam       0.97      1.00      0.98       966
       Spam       1.00      0.78      0.88       149

   accuracy                           0.97      1115

[[966   0]
 [ 33 116]]
```

[Insert screenshot of the program running here]

Result Interpretation

• Accuracy 97.04%: about 97 of every 100 test messages were classified correctly.

• Spam precision 1.00: no legitimate message was marked as spam (0 false positives).

• Spam recall 0.78: 116 of 149 spam messages were caught and 33 were missed. This is the main weakness.

Sample Messages Tested

Sample Message	**Result**
Congratulations! You have won a free prize of 1000 pounds. Call now to claim	SPAM MESSAGE
Hey, are we meeting at the library at 5 today?	NOT SPAM MESSAGE
URGENT! Your account is blocked. Click the link to verify and win cash	SPAM MESSAGE

7. CONCLUSION

The SMS Spam Detection system was built with Python, TF-IDF and Multinomial Naive Bayes. It reached 97.04% accuracy on 1,115 unseen messages, never blocked a legitimate message, and can also check messages from screenshots using OCR.

However, it missed 33 of 149 spam messages (recall 78%), the dataset is old and English-only so new scams and Hindi or Marathi messages may be misclassified, and blurry screenshots give poor OCR text. In future, the project can use a newer multilingual dataset, tune class weights to improve spam recall, try models such as SVM or transformers, and add a web interface.

REFERENCES

• Almeida, T. A., Gómez Hidalgo, J. M., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. Proceedings of the 11th ACM Symposium on Document Engineering (DocEng ’11), 259–262.

• Dua, D., & Graff, C. UCI Machine Learning Repository: SMS Spam Collection. University of California, Irvine. https://archive.ics.uci.edu/dataset/228/sms+spam+collection

• McCallum, A., & Nigam, K. (1998). A comparison of event models for Naive Bayes text classification. AAAI-98 Workshop on Learning for Text Categorization, 41–48.

• Pang, B., & Lee, L. (2008). Opinion mining and sentiment analysis. Foundations and Trends in Information Retrieval, 2(1–2), 1–135.

• Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

• Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523.

• Smith, R. (2007). An overview of the Tesseract OCR engine. Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007), 629–633.